

GGSEC RESEARCH · WHITEPAPER

UEFI Attack Surface Research

Bootkits · Secure Boot bypasses · Pre-OS exploitation vectors ·
Firmware rootkit detection · SMM · DXE

Author: Maciej Gojny

Organization: GG Advanced IT Security

Web: ggsec.de

Version: 1.1 · July 2026

TLP:WHITE — PUBLIC RELEASE

01 Executive Summary

The Unified Extensible Firmware Interface (UEFI) has replaced legacy BIOS as the industry-standard firmware interface for x86, x64, and ARM platforms. While UEFI brings significant security improvements — Secure Boot, cryptographic verification, and a standardized architecture — it also introduces a dramatically expanded attack surface.

UEFI firmware executes before the operating system, before any security agent loads, and before most hardware-based attestation begins. A compromise at this layer provides the attacker with:

- **Persistence** that survives OS reinstalls, disk replacements, and factory resets
- **Stealth** invisible to all OS-level security tools (EDR, AV, HIDS)
- **Privilege** equivalent to hypervisor or System Management Mode (SMM)
- **Control** over boot process, Secure Boot policy, and hardware initialization

Key Insight: A UEFI bootkit is the cybersecurity equivalent of a building tenant changing the locks and then giving you a key that only works when they allow it. You never truly own your system again.

1.1 Business Impact

Impact Area	Consequences
Operational	Persistent compromise across fleet; remediation requires physical reflashing or motherboard replacement
Financial	Firmware incidents carry materially higher remediation cost than OS-level compromise, driven by per-device physical handling, specialist tooling and possible hardware replacement. GGSEC does not publish a point estimate; scope cost against fleet size and device accessibility.
Compliance	PCI DSS, HIPAA, FedRAMP violations due to inability to attest boot integrity
Reputational	Loss of customer trust when bootkits are discovered in production environments

02 UEFI Architecture Overview

2.1 UEFI Boot Phases

UEFI BOOT PHASES – ATTACK SURFACE

SEC Phase

PEI Phase

DXE Phase

CRITICAL — largest attack surface

BDS Phase

TSL Phase

RT Phase

03 Bootkit Architecture & Capabilities

3.1 What is a UEFI Bootkit?

A UEFI bootkit is malicious code that resides in the SPI flash storage (firmware) and executes during the UEFI boot process, before the OS loads. Unlike traditional bootkits that resided on disk (MBR/VBR) and were removed during OS reinstallation, UEFI bootkits survive all software-based remediation.

3.2 Bootkit Insertion Vectors

Insertion Vector	Privilege Required	Persistence	Detectability
SPI flash write	Kernel/System (Ring 0)	Permanent until reflash	Medium
UEFI capsule update	Admin/Platform owner	Permanent	Low
Runtime variable	Kernel (Ring 0)	Survives reboot	High
Option ROM	PCIe device vendor	Per-device	Medium

Insertion Vector	Privilege Required	Persistence	Detectability
SMM callout	Ring 0 + SMI handler	Firmware persistent	Low

3.3 Bootkit Capability Matrix

Capability	MBR Bootkit	UEFI Bootkit	UEFI Firmware Bootkit
Survives OS reinstall	✓	✓	✓
Persists after disk format	✗	✓	✓
Survives firmware update	✗	✗	✓ (if not properly signed)
Invisible to EDR/AV	Partial	✓	✓
Can subvert Secure Boot	✗	✓	✓
Requires physical reflash	✗	✗	✓
OS-level detection possible	Partial	Limited	Near impossible

04 Secure Boot Bypass Techniques

4.1 How Secure Boot Works

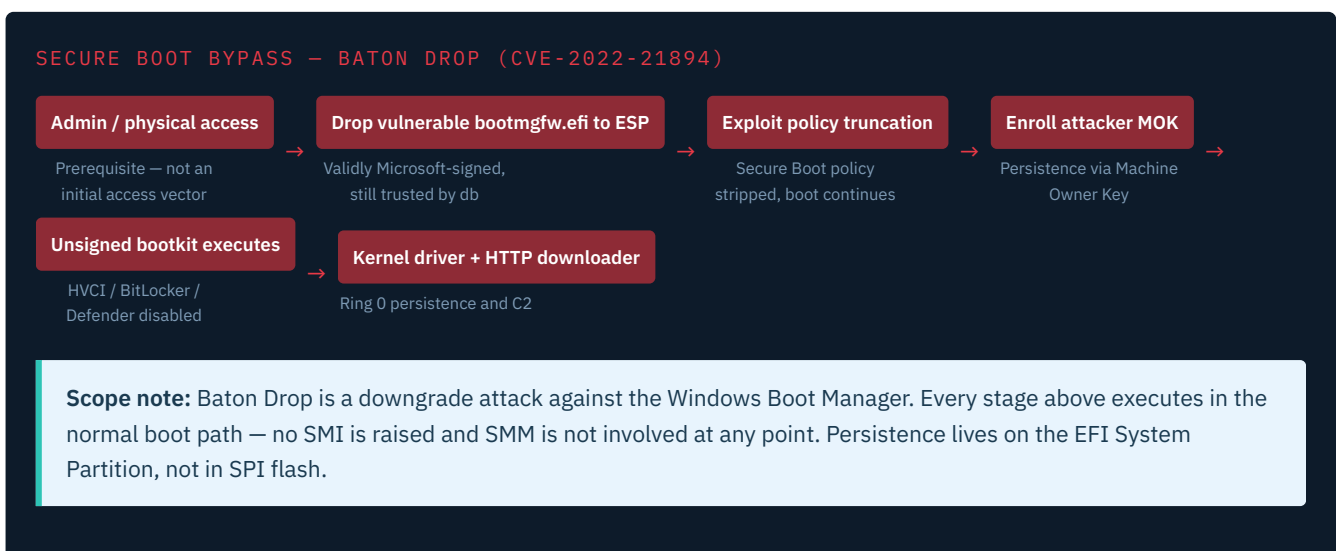
Secure Boot ensures that only cryptographically signed and trusted software executes during the boot process. The UEFI firmware contains Platform Key (PK) and Key Exchange Keys (KEK). The signature database (db) contains authorized signatures or hashes. The forbidden database (dbx) contains revoked signatures. Each binary (bootloader, driver, option ROM) is verified before execution.

4.2 Known Secure Boot Bypasses

Bypass Technique	CVE	Mechanism	Mitigation
Golden Key	MS16-094 / MS16-100 (2016)	Leak of a Microsoft Secure Boot <i>debug/supplemental policy</i> allowing policy enforcement to be disabled. Not a master-key leak and not tracked under CVE-2018-12181.	Policy revocation via dbx; boot manager updates
BootHole	CVE-2020-10713	Buffer overflow in the GRUB2 <code>grub.cfg</code> flex/bison configuration parser, leading to arbitrary code execution in the boot chain	Update dbx with vulnerable GRUB hashes
Baton Drop	CVE-2022-21894	Secure Boot Security Feature Bypass in signed copies of the Windows Boot Manager. The bootloader truncates the Secure Boot policy values and continues booting instead of halting on missing DB/DBX data. Not an SMM issue.	DBX revocation of vulnerable boot managers (KB5025885 chain)

Bypass Technique	CVE	Mechanism	Mitigation
Secure Boot bypass (follow-on)	CVE-2023-24932	Further Secure Boot bypass addressed by Microsoft in May 2023; the fix required manual opt-in activation, delaying real-world protection	Apply KB5025885 and complete the opt-in enforcement steps
PKfail	CVE-2024-8105	Hundreds of OEM device models shipped with AMI <i>test</i> Platform Keys explicitly marked “DO NOT TRUST”; the private keys were subsequently leaked publicly. Disclosed by Binarly, July 2024.	Replace test PKs with vendor production keys; audit PK/KEK/db provenance
LogoFAIL	CVE-2023-40238 (and related)	Image parser vulnerabilities in DXE boot-logo handling. LogoFAIL is a vulnerability <i>class</i> spanning multiple CVEs across Insyde, AMI and Phoenix.	Patch UEFI firmware, driver updates

4.3 Secure Boot Bypass Attack Flow



4.4 Detection of Secure Boot Bypass

Indicator	Detection Method	Tooling
Secure Boot disabled unexpectedly	UEFI variable monitoring	sbctl, bootinfoscript
Unexpected certificates in db	Certificate inventory comparison	sbctl, mokutil
Missing expected certificates	Baseline comparison	Custom inventory scripts
Forged SetupMode variable	Variable integrity check	TPM attestation
SMI handler corruption	SMI handler validation	Chipsec, SMM detection tools

5.1 DXE Driver Injection

The Driver Execution Environment (DXE) loads hundreds of UEFI drivers during boot. Each driver represents a potential attack surface.

Attack Method: Compromise running OS (kernel privilege), write malicious DXE driver to SPI flash or EFI System Partition (ESP), modify boot order or driver load order, then malicious driver loads during next boot before any security software.

Detection: Firmware integrity scanning (e.g., CHIPSEC), driver hash verification against golden images, boot-time driver enumeration logging.

5.2 UEFI Runtime Variable Attacks

UEFI variables store configuration data in NVRAM. Attackers can modify BootOrder to load malicious bootloader first, set SetupMode=1 to allow unsigned code, inject Boot#### variables pointing to attacker-controlled binaries, or overwrite SecureBoot=Disabled in variable storage.

Forensic Artifacts: Unexpected BootOrder values, unknown Boot#### variables, SecureBoot state changes without authorized update, AuditMode or DeployedMode transitions.

5.3 Option ROM Attacks

PCIe devices (NICs, storage controllers, GPUs) can ship with Option ROMs containing UEFI drivers. Attackers can flash malicious Option ROM to PCIe device, leverage DMA capabilities to read/write system memory, and execute code in UEFI context during device initialization.

Detection: Option ROM scanning and hash verification against vendor database.

5.4 ACPI Table Manipulation

ACPI tables provide power management and hardware configuration. Malicious modifications can inject code via DSDT (Differentiated System Description Table), manipulate FACP (Fixed ACPI Description Table) for SMM abuse, or add malicious SSDT (Secondary System Description Table) entries.

Tooling: acpiexec, iasl for table inspection.

5.5 SMM (System Management Mode) Subversion

SMM is the highest privilege execution environment (Ring -2). SMM code executes in response to System Management Interrupts (SMIs) and can access all system memory.

Attack Vectors: SMI handler buffer overflow leading to SMM code execution, SMM callout via runtime service, SMM cache poisoning, and software-generated SMI with attacker-controlled parameters.

Detection: SMM integrity monitoring via CHIPSEC module common.smm, SMI handler enumeration and analysis, memory scanning for SMM-resident implants.

06 Known UEFI Firmware Rootkits & Malware

6.1 Comprehensive Malware Database

Malware	Discovered	Target	Mechanism	Persistence
BlackLotus	2022 (ESET analysis Mar 2023)	Windows 10/11	Bypasses Secure Boot via CVE-2022-21894 (Baton Drop) using a validly-signed but vulnerable boot manager	ESP (bootkit, not a firmware implant)
MoonBounce	2021/2022	Windows	Modifies the pre-existing CORE_DXE component in AMI firmware rather than adding a new module	SPI flash (firmware implant)
FinSpy / FinFisher	2021 disclosure	Windows	Commercial surveillance suite; UEFI bootkit variant replacing the Windows Boot Manager	ESP
LoJax (Sednit / APT28)	2018	Windows	First UEFI rootkit confirmed in the wild; built on a repurposed Absolute Computrace/LoJack module	SPI flash
MosaicRegressor	2020	Windows	Based on the leaked Hacking Team Vector-EDK implant framework; drops a multi-stage downloader at each boot	SPI flash
CosmicStrand	2022	Windows	Kernel hook injected via the CSMCORE driver; targets GIGABYTE/ASUS H81 consumer boards	SPI flash
ESpecter	2021	Windows	Malicious bootloader on ESP	EFI System Partition
Bootkitty	2024	Linux (Ubuntu)	Proof-of-concept UEFI bootkit for Linux. Self-signed — requires the operator to enroll their own MOK; it is not trusted by default Secure Boot.	ESP

6.2 BlackLotus Deep Dive

BlackLotus (2022-2023) is the most sophisticated publicly documented UEFI bootkit.

Attribute	Detail
Discovery	Advertised on underground forums from October 2022; first public analysis published by ESET Research, March 2023
CVE	CVE-2022-21894 — “Baton Drop”, a Secure Boot Security Feature Bypass in the Windows Boot Manager
Target	Windows 10/11 x64 with Secure Boot enabled
Prerequisite	Not an initial access vector — requires prior privileged or physical access to the device
Capabilities	Disables HVCI, BitLocker and Microsoft Defender Antivirus; loads an unsigned kernel driver and a user-mode HTTP downloader for C2

Attribute	Detail
Persistence	EFI System Partition (ESP) plus enrollment of the attacker's Machine Owner Key (MOK). BlackLotus does not write to SPI flash — it is a bootkit, not a firmware implant.
Detection evasion	Kernel driver protects bootkit files on the ESP from removal
Forensic artifacts	Unexpected paths under <code>\EFI\Microsoft\Boot\</code> ; mismatched ESP file modification timestamps versus <code>C:\Windows\Boot\EFI</code>
Remediation	ESP wipe and OS reinstall are sufficient. Physical SPI reflash is not required for BlackLotus alone.
Distribution	Underground forums, approximately US\$5,000 per license

Mitigation: Apply Microsoft KB5025885 (2023) and subsequent patches and *complete the opt-in enforcement steps* — the initial release did not enforce revocation automatically. Update DBX to revoke the vulnerable boot managers, and enable HVCI plus TPM-backed measured boot.

6.3 MoonBounce Deep Dive

MoonBounce (discovered by Kaspersky, 2021) targeted AMI UEFI firmware on enterprise laptops.

Attribute	Detail
Target	AMI UEFI firmware on enterprise laptops
Insertion	Modification of the pre-existing <code>CORE_DXE</code> component in AMI firmware. No new module is added — this is what makes MoonBounce substantially harder to detect than LoJax.
Persistence	Firmware-level; survives disk replacement
Detection	Byte-level comparison of <code>CORE_DXE</code> against a known-good vendor image; no new files appear on the ESP, so ESP-based detection fails entirely
Attribution	APT41 (suspected)

6.4 OEM Persistence Modules — The Legitimate Backdoor Class

A category distinct from malware deserves separate treatment: vendor-supplied firmware modules that implement the same technical primitives as an implant, but ship from the factory. They are legitimate by intent and indistinguishable from an implant by behaviour. Threat actors have repurposed them directly.

Module	Vendor	Mechanism	Security relevance
Computrace / LoJack	Absolute Software	UEFI module drops <code>ipcnetsp.exe</code> to System32 by replacing <code>AUTOCHK.EXE</code> ; installs a Windows service; C2 to vendor infrastructure	Repurposed by APT28 as LoJax — the first in-the-wild UEFI rootkit. Factory-embedded on a reported 600M+ devices.
Lenovo Service Engine (LSE)	Lenovo	WPBT-based; replaces <code>autochk.exe</code> to reinstate <code>LenovoUpdate.exe</code> / <code>LenovoCheck.exe</code> after a clean OS install	Withdrawn 2015 after buffer overflow disclosure; required a BIOS update to remove

Module	Vendor	Mechanism	Security relevance
App Center Download & Install	GIGABYTE	<code>WpbtDxe.efi</code> drops an embedded Windows native binary that fetches payloads over plain HTTP with no signature validation	271 board models, ~7M devices (Eclipsium, 2023). MITM-exploitable by design.
Vendor password recovery	Various (ASUS, HP)	Undocumented service paths permitting UEFI setup or HDD password reset without authentication	Patched individually; illustrates the class rather than any single case

Detection guidance: the reliable structural signal is a Windows-subsystem PE (Subsystem 1 = Native, or 2 = GUI) embedded inside a UEFI-subsystem module (Subsystem 10–12). GGSEC's **ggfw** toolkit (§13.1) flags this pattern regardless of vendor attribution. Supporting indicators: references to the WPBT ACPI table; the strings `autochk.exe` , `AUTOCHK.BAK` , `BootExecute` ; and plain-HTTP update URLs inside DXE modules.

Assessment note: a dormant OEM module is not an active compromise, but it is not zero-risk either. Where SPI descriptor access control permits host writes to the BIOS region, an attacker with local privilege can reactivate or replace such a module. Report the descriptor permission finding as the primary issue and the OEM module as context.

07 Detection Methodologies

7.1 Firmware Integrity Scanning Tools

Tool	Method	Strengths	Limitations
CHIPSEC	Platform analysis framework	Comprehensive SMM, variable, SPI checks	Requires technical expertise
BinWalk	Firmware binary analysis	Extracts and parses UEFI volumes	Labor-intensive, no automation
UEFITool	UEFI image manipulation/analysis	GUI for firmware exploration	Manual analysis only
Fwhunt	UEFI module analysis	Detects known malicious patterns	Signature-based
ggfw (GGSEC)	Automated offline SPI analysis and platform posture assessment	Single-pass descriptor, FV, variable-store and PE/DXE analysis with confidence-scored findings; PE extraction for YARA triage; live MEI fallback where offline evidence is insufficient	Heuristic triage rather than a signature engine; in beta

7.2 CHIPSEC Audit Modules

```
# Basic system audit

python chipsec_main.py -a -l chipsec.log


# Critical UEFI audit modules

python chipsec_util.py uefi scan

python chipsec_util.py spi dump firmware.bin

python chipsec_util.py smi scan

python chipsec_util.py vars list


# Secure Boot verification

python chipsec_util.py uefi secureboot


# SMM integrity check

python chipsec_util.py smm info

python chipsec_util.py smm compress
```

7.3 Detection Analytics (MITRE ATT&CK)

Technique	ID	Detection Method
System Firmware	T1542.001	Firmware version monitoring; SPI flash hash comparison
Bootkit	T1542.003	UEFI variable monitoring; boot order baseline
Secure Boot Bypass	T1553.001	Signature verification events; db/dbx modification audit
SMM Abuse	T1055 (variant)	SMI handler integrity; SMM communication monitoring
UEFI Variable Manipulation	T1612	Runtime variable audit logging

7.4 Enterprise Detection Controls

Control	Implementation	Coverage
Firmware inventory	Periodic SPI flash hash collection (PowerShell + WMI)	Baseline deviation
Secure Boot monitoring	Event ID 1035 (Secure Boot validation failure)	Windows Event Log
UEFI variable audit	Get-UEFIVariable PowerShell logging	Variable changes

Control	Implementation	Coverage
Boot order baseline	Centralized boot order registry	Unexpected changes
TPM attestation	PCR 0-7 measurements; remote attestation	Hardware-backed integrity

7.5 Windows-Specific Detection

```
# Check Secure Boot status

Confirm-SecureBootUEFI

Get-SecureBootPolicy


# Enumerate UEFI variables

Get-UEFIVariable -Name "BootOrder"

Get-UEFIVariable -Name "SecureBoot"


# Check for unexpected boot entries

bcdedit /enum all


# Review Code Integrity events

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-CodeIntegrity/Operational'; ID=3076}
```

7.6 Linux-Specific Detection

```
# Check Secure Boot status

sbctl status

mokutil --sb-state


# Enumerate UEFI variables

ls /sys/firmware/efi/efivars/

efivar -l


# Verify kernel signature

kmodsign --verify /boot/vmlinuz-$(uname -r)


# Firmware detection (fwupd)

fwupdmgr get-devices

fwupdmgr get-updates
```

08 Mitigation Strategies

8.1 Architectural Controls

Control	Description	Priority
Secure Boot enforcement	Mandatory signature verification; UEFI 2.3.1+	Critical
Firmware write protection	SPI flash write protect via hardware jumper or BIOS setting	Critical
Measured Boot (TPM)	Boot measurements in PCRs 0-7; remote attestation capability	Critical
UEFI capsule updates signed	Require manufacturer-signed firmware updates	High
SMM isolation	SMM page table protection; restricted SMI sources	High
Runtime variable protection	Critical variables locked after EndOfDxe	Medium
Option ROM verification	Signature check for PCIe Option ROMs	Medium

8.2 Microsoft Recommended Mitigations for BlackLotus / Baton Drop

KB	Date	Mitigation	Deployment
KB5025885	Apr 2023	Initial BlackLotus mitigation	Windows 10/11
KB5027455	Jun 2023	Enhanced dbx update	Windows 10/11
KB5025885 (update)	Aug 2023	Final revocations; full BlackLotus blocking	All supported Windows

```
# Deployment verification

reg query HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\Changes

reg query HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\Updated
```

8.3 Vendor Firmware Update Best Practices

- Enable automatic firmware updates from trusted vendors for timely security patch deployment
- Verify checksums before flashing to prevent corrupted/malicious updates
- Use hardware-backed update verification to prevent rollback attacks
- Maintain recovery image for rapid restoration if compromise detected
- Disable legacy BIOS compatibility (CSM) to reduce attack surface

8.4 Incident Response for Bootkit Compromise

When bootkit is suspected, follow these steps:

1. **Isolate affected system** – Assume all data is compromised
2. **Capture forensic artifacts** – SPI flash dump (hardware programmer preferred), UEFI variable export, TPM event log, Secure Boot event logs
3. **Determine persistence mechanism** – SPI flash vs. ESP vs. Option ROM
4. **Remediate** – For SPI flash compromise: physical reflash with verified image or motherboard replacement. For ESP compromise: secure wipe and OS reinstall. For Option ROM compromise: replace affected PCIe device
5. **Verify remediation** – CHIPSEC audit, Secure Boot re-verification
6. **Root cause analysis** – How did initial compromise occur?

09 Forensic Analysis

9.1 SPI Flash Acquisition

Physical acquisition (preferred): Identify SPI flash chip (Winbond, Macronix, etc.), connect programmer (CH341A, Dediprog, Bus Pirate), dump full flash contents (8MB-32MB typical).

Software acquisition (limited): CHIPSEC (`chipsec_main.py -a -l log.txt`) and UEFITool for runtime extraction. Note that a host-side dump may return a blank ME region where descriptor permissions deny the read — full image size does not imply full image content.

Analysis: Binary diff against known-good image, UEFITool volume extraction, disassemble suspicious DXE or SMM modules, check for unexpected PEI or DXE drivers.

9.2 Forensic Artifacts

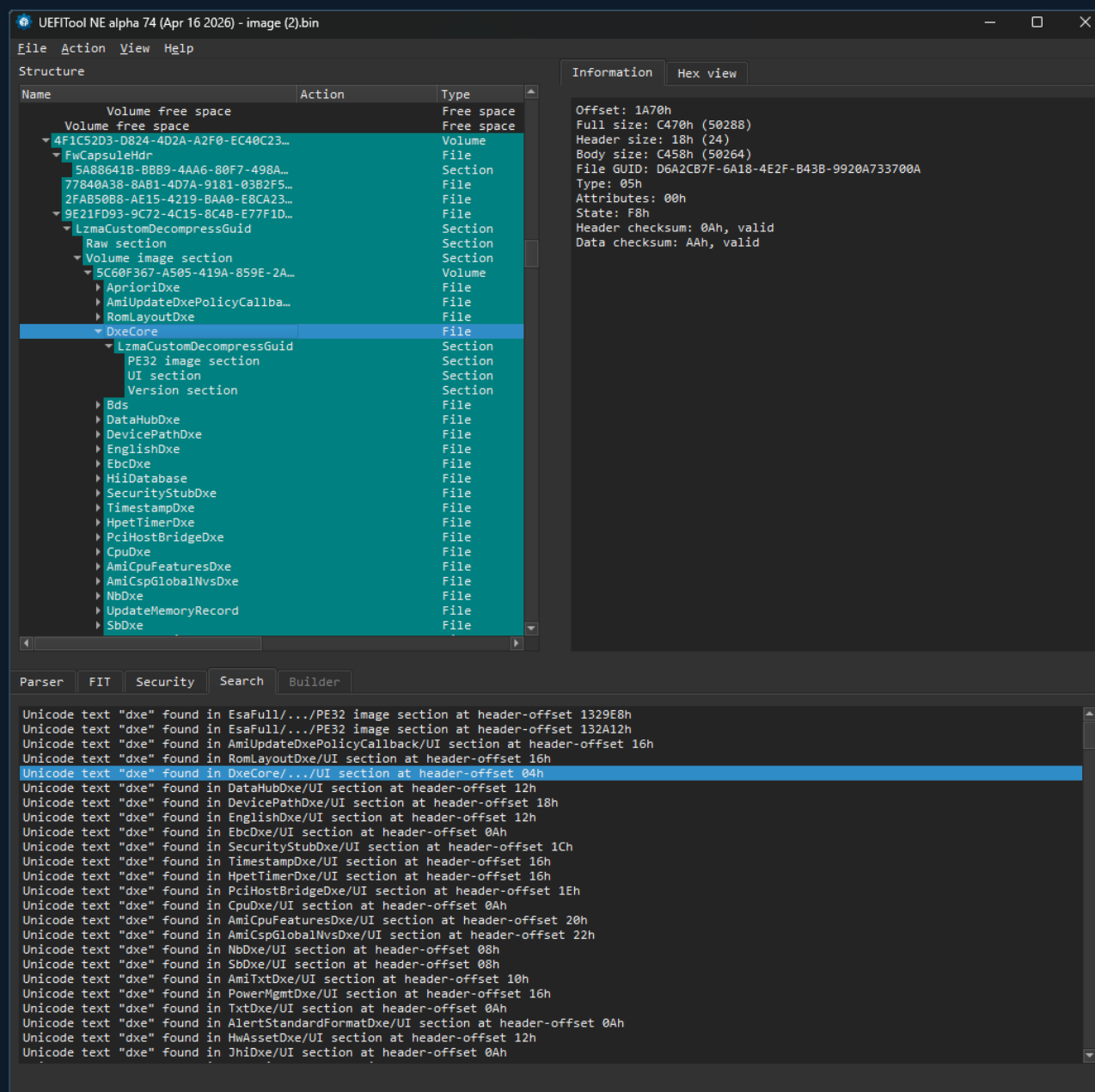
Artifact	Location	Forensic Value
SPI flash image	Physical flash chip	Complete firmware; all modifications visible
UEFI variables	NVRAM (part of SPI flash)	Boot order, Secure Boot state, db/dbx
EFI System Partition (ESP)	Disk partition (FAT32)	Bootloaders, boot applications, drivers
TPM event log	TPM hardware / OS log	Boot measurements; attestation failures
Secure Boot event log	Windows Event Log	Signature failures; revoked binaries

9.3 Signature-Based Detection (Blacklist)

Indicator	Type	Source
BlackLotus SPI patterns	Hash	ESET, Microsoft
MoonBounce DXE module	Hash	Kaspersky
Compromised GRUB versions (BootHole)	Hash list	Microsoft dbx, US-CERT
Test PK certificates	Certificate	CVE-2023-24932

9.4 Practical Firmware Analysis – Real-World Artifacts

FIG 1 – UEFITool NE – VOLUME 000: DXE DRIVERS (DXECORE, APRIORIDX, SECURITYSTUBDXE)



DxeCore, AprioriDxe, SecurityStubDxe – key modules for analysis

FIG 2 – EXTRACTED DXE MODULE SET – AMI FIRMWARE

Name	Date modified	Type
File 000 - AprioriDxe	04.06.2026 13:12	File folder
File 001 - AmiUpdateDxePolicyCallback	04.06.2026 13:12	File folder
File 002 - RomLayoutDxe	04.06.2026 13:12	File folder
File 003 - DxeCore	04.06.2026 13:12	File folder
File 004 - Bds	04.06.2026 13:12	File folder
File 005 - DataHubDxe	04.06.2026 13:12	File folder
File 006 - DevicePathDxe	04.06.2026 13:12	File folder
File 007 - EnglishDxe	04.06.2026 13:12	File folder
File 008 - EbcDxe	04.06.2026 13:12	File folder
File 009 - HiiDatabase	04.06.2026 13:12	File folder
File 010 - SecurityStubDxe	04.06.2026 13:12	File folder
File 011 - TimestampDxe	04.06.2026 13:12	File folder
File 012 - HpetTimerDxe	04.06.2026 13:12	File folder
File 013 - PciHostBridgeDxe	04.06.2026 13:12	File folder
File 014 - CpuDxe	04.06.2026 13:12	File folder
File 015 - AmiCpuFeaturesDxe	04.06.2026 13:12	File folder
File 016 - AmiCspGlobalNvsDxe	04.06.2026 13:12	File folder
File 017 - Nbdxe	04.06.2026 13:12	File folder
File 018 - UpdateMemoryRecord	04.06.2026 13:12	File folder
File 019 - Sbdxe	04.06.2026 13:12	File folder
File 020 - UpdateSmbiosType41	04.06.2026 13:12	File folder
File 021 - I2cHcOperation	04.06.2026 13:12	File folder
File 022 - SpiHcOperation	04.06.2026 13:12	File folder
File 023 - NvmeRstPassword	04.06.2026 13:12	File folder
File 024 - SkipStoragePwdPrompt	04.06.2026 13:12	File folder
File 025 - AmiTxtDxe	04.06.2026 13:12	File folder
File 026 - PowerMgmtDxe	04.06.2026 13:12	File folder
File 027 - TxtDxe	04.06.2026 13:12	File folder

Per-module extraction output: AprioriDxe, AmiUpdateDxePolicyCallback, RomLayoutDxe, DxeCore, Bds, DataHubDxe, DevicePathDxe, EbcDxe, HiiDatabase, SecurityStubDxe, CpuDxe, Nbdxe, Sbdxe, SpiHcOperation and others — each becomes an independent hashing and analysis unit

FIG 3 – ME ANALYZER – FTPR PARTITION: KERNEL, CRYPTO, PTT (PLATFORM TRUST TECHNOLOGY)

ME Analyzer v1.311.0 r377

Partition Name	FTPR
Checksum	0xCD10E0E0

SCP Checksum of partition "FTPR" is VALID

Detected 29 Module(s) at FTPR 0001 [0x063000]

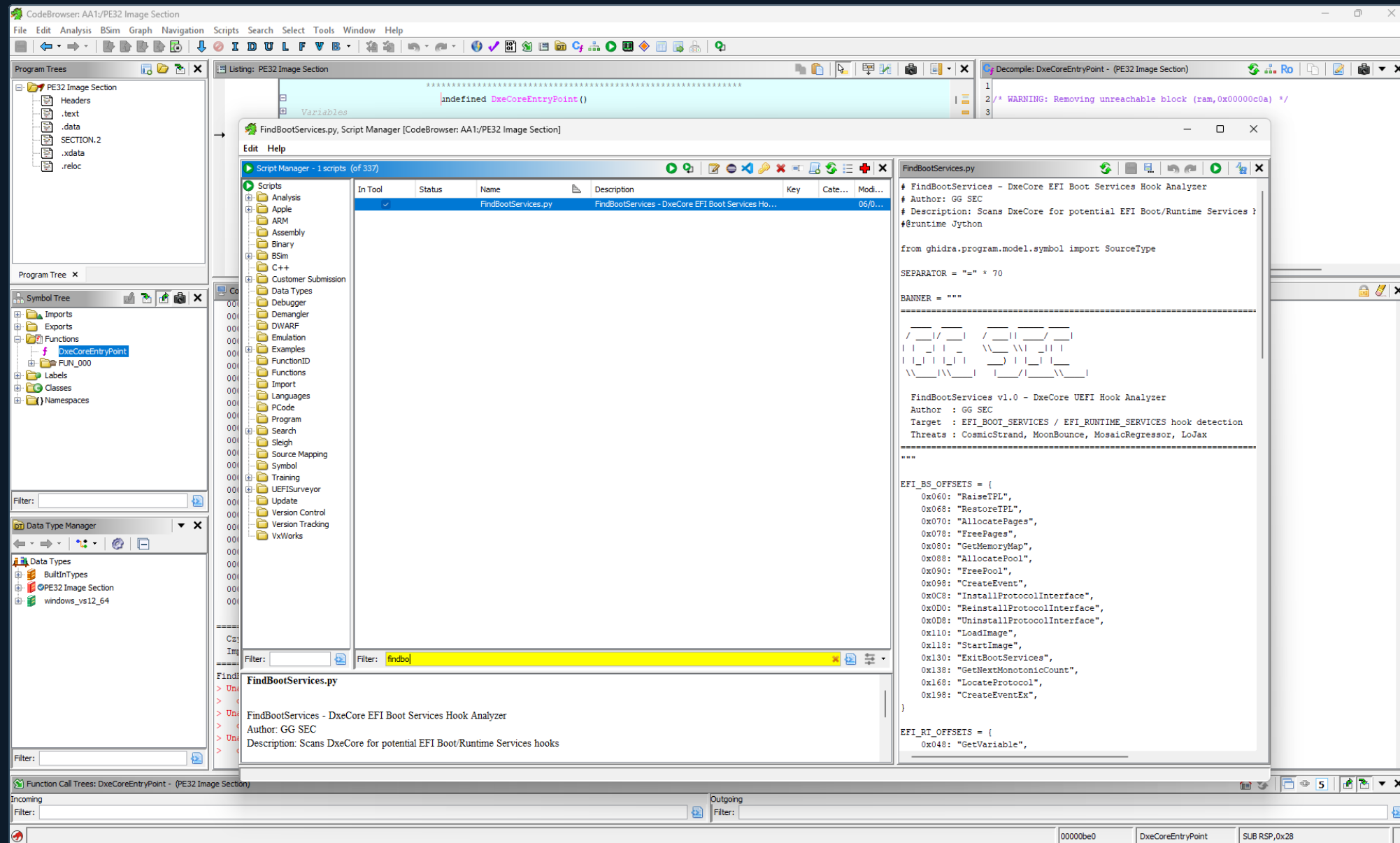
Name	Compression	Encryption	Offset	Compressed	Uncompressed	Empty
FTPR.man	None	None	0x0632CC	0x000574	0x000574	No
intl.cfg.met	None	None	0x063840	0x000048	0x000048	No
intl.cfg	None	None	0x0CD000	0x00497F	0x00497F	No
rot.key	None	None	0x064000	0x000798	0x000798	No
fitc.cfg	None	None	0x063000	0x000000	0x000000	Yes
kernel	Huffman	None	0x065000	0x013000	0x01A000	No
syslib	Huffman	None	0x078000	0x01C000	0x025000	No
bup	Huffman	None	0x094000	0x039000	0x04E000	No
pm	Huffman	None	0x0D2000	0x003000	0x004000	No
vfs	Huffman	None	0x0D5000	0x011000	0x017000	No
evtdisp	Huffman	None	0x0E6000	0x003000	0x004000	No
loadmgr	Huffman	None	0x0E9000	0x005000	0x007000	No
busdrv	Huffman	None	0x0EE000	0x002000	0x002000	No
prtc	Huffman	None	0x0F0000	0x001000	0x002000	No
smbus	LZMA	None	0x0F1000	0x002077	0x004000	No
crypto	Huffman	None	0x0F4000	0x028000	0x036000	No
fpf	Huffman	None	0x11C000	0x004000	0x005000	No
storage	Huffman	None	0x120000	0x00CB40	0x012000	No
gpio	Huffman	None	0x12CB40	0x0017C0	0x002000	No
ipc_drv	Huffman	None	0x12E300	0x0025C0	0x004000	No
sec_msg	Huffman	None	0x1308C0	0x0008C0	0x001000	No
policy	Huffman	None	0x131180	0x006200	0x009000	No
heci	Huffman	None	0x137380	0x006300	0x009000	No
pmdrv	Huffman	None	0x13D680	0x001D40	0x003000	No
maestro	Huffman	None	0x13F3C0	0x002780	0x004000	No
fwupdate	Huffman	None	0x141B40	0x006000	0x009000	No
ptt	Huffman	None	0x147B40	0x01D500	0x029000	No
mca_boot	Huffman	None	0x165040	0x002580	0x004000	No
mca_srv	Huffman	None	0x1675C0	0x030A40	0x007000	No

→ Stored Uncompressed metadata "FTPR.man" [0x0632CC - 0x06383F]

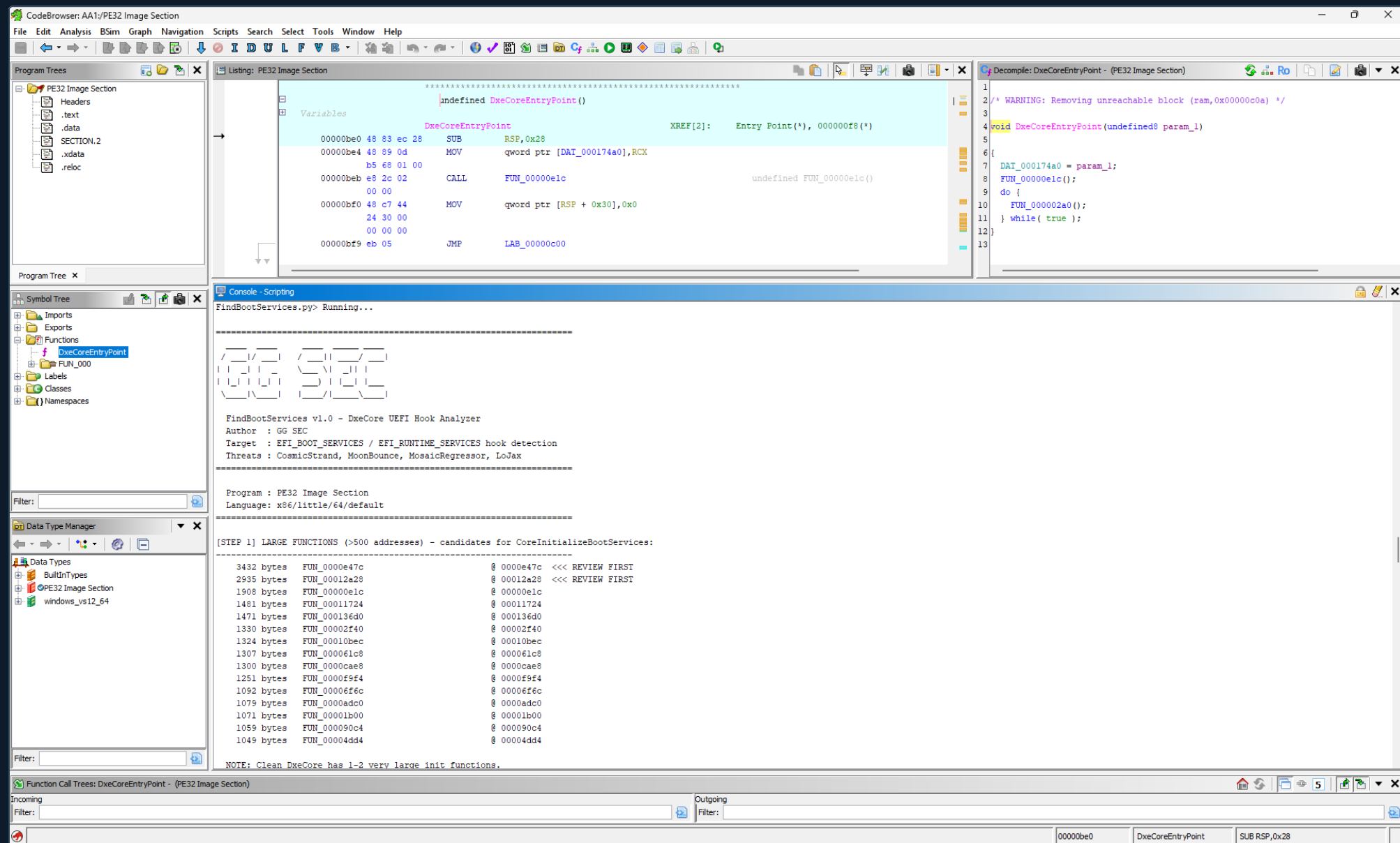
→ Stored Uncompressed metadata "FTPR.man" [0x0632CC - 0x06383F]

kernel (76KB), crypto (216KB), ptt (164KB – software TPM), fwupdate, heci

FIG 4 – GHIDRA – DXECOREENTRYPOINT + FINDBOOTSERVICES.PY (GG SEC HOOK DETECTOR)

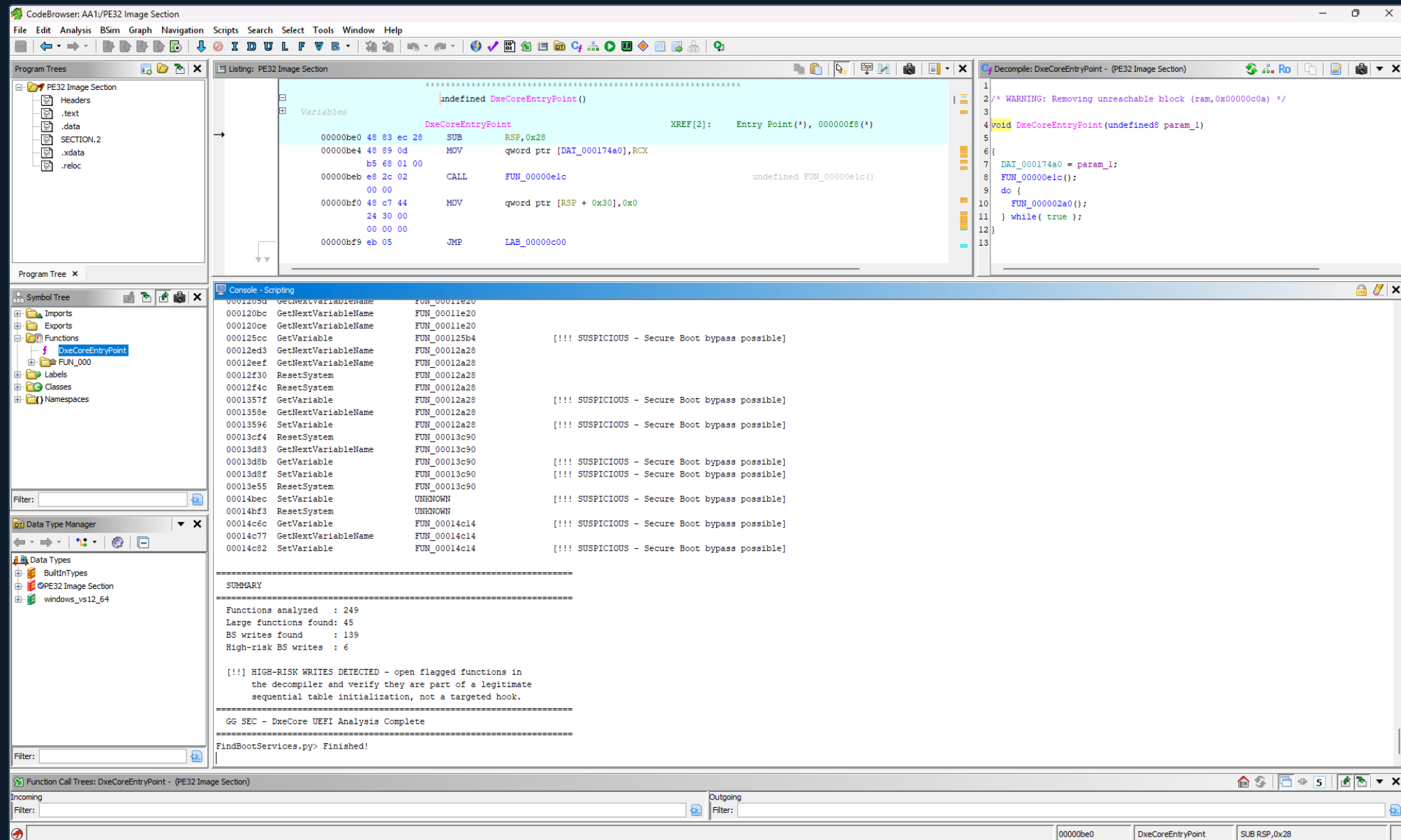


GG SEC custom script: FindBootServices – scans DxeCore for EFI_BOOT_SERVICES hooks



Candidate identification for CoreInitializeBootServices, ranked by function size. A clean DxeCore is expected to contain one or two very large initialization functions — a deviation from that shape is itself a signal

FIG 6 – GHIDRA – FINDBOOTSERVICES.PY, SCAN RESULTS AND SUMMARY



GetVariable / SetVariable writes into the runtime services table flagged for review. Summary for this image: 249 functions analysed, 45 large functions, 139 boot-services writes, 6 high-risk. The tool explicitly instructs the analyst to verify whether flagged writes are sequential table initialization rather than a targeted hook

10 DXE Driver Deep Dive – BiosGuard, Hooking & Boot Services Manipulation

10.1 DXE Driver Loading Order – AprioriDxe

Driver	Purpose	Security Relevance
DataHubDxe	Data hub for system information	Low – info gathering
CpuDxe	CPU initialization, SMM setup	HIGH – SMM entry point
PciHostBridgeDxe	PCI host bridge	Medium – DMA attack surface

10.2 BiosGuardDxe – Critical Security Driver

Attack surface: BiosGuardDxe mediates authenticated flash writes. Research has demonstrated Boot Guard / BIOS Guard bypasses achieved by manipulating the Hand-Off Blocks (HOBs) that carry verification state between boot phases. *Citation caution:* CVE-2017-5706 is sometimes associated with this class in secondary sources, but it belongs to INTEL-SA-00086 (buffer overflows in Intel Server Platform Services firmware). Confirm any HOB bypass CVE against the primary vendor advisory before citing it.

10.3 Critical DXE Drivers for Security Research

Driver	Why Important
DxeCore	PRIME TARGET – CosmicStrand, MosaicRegressor hook DxeCore
SmmCore / *Smm*	HIGHEST PRIVILEGE (Ring -2) – target of SMI handler vulnerability classes such as CVE-2020-24489. <i>Not</i> a BlackLotus target; BlackLotus does not interact with SMM.
BiosGuardDxe	Bypass = unrestricted flash write

10.4 EFI Boot Services Hooking

Service	When Hooked	Known Malware
ExitBootServices	Execute payload before OS	CosmicStrand
AllocatePages	Allocate hidden memory	MoonBounce
CreateEventEx	Schedule malicious callback	MosaicRegressor

10.5 Indicators of Compromise (IoC) – DXE Level

Indicator	Typical of
Write to gBS→ExitBootServices pointer	CosmicStrand
CreateEventEx with unknown GUID	MosaicRegressor

Indicator	Typical of
Jump to address outside PE range	Generic implant indicator (MoonBounce, MosaicRegressor)

11 SMM Deep Dive – Ring -2 & System Management Mode

System Management Mode (SMM) is the highest privilege level in x86/x64 – **Ring -2**, beyond hypervisor (Ring -1) and kernel (Ring 0). SMM code executes in response to System Management Interrupts (SMIs) and can access all system memory, including hypervisor and kernel space.

Why SMM matters: If an attacker compromises SMM, no software-based detection (EDR, AV, hypervisor) can see or stop them. SMM is invisible from the OS.

11.1 SMM Architecture – Attack Surface

Component	Attack Surface
SMI (System Management Interrupt)	Malicious SMI generation → forced entry
SMI Handler	Buffer overflow → arbitrary SMM code
SMRAM (SMM RAM)	SMRAM cache poisoning, lock bypass
SMM Communication Protocol	Buffer manipulation → SMM memory corruption
SW SMI (I/O port 0xB2)	Unprivileged SMI generation → DoS / code execution

11.2 Documented SMI Handler Vulnerability Classes

Scope note: SMM attack classes are frequently conflated with Secure Boot bypasses in public reporting. They are distinct. Boot manager bypasses such as CVE-2022-21894 execute in the normal boot path and never enter SMM. The classes below describe genuine Ring -2 attack surface.

Class	Mechanism	Representative CVE
SMI handler buffer overflow	Attacker-controlled data in the SMM communication buffer overruns handler-local storage, yielding code execution in SMRAM	CVE-2020-24489
SMM callout	An SMI handler dereferences a pointer into memory outside SMRAM that the attacker controls, executing non-SMRAM code at Ring -2	Multiple vendor-specific
SMRAM lock bypass	D_LCK / SMRR left unset, permitting direct SMRAM access from Ring 0	Configuration weakness
SMM cache poisoning	MTRR manipulation causes SMRAM fetches to resolve from attacker-controlled cache lines	Legacy platforms

11.3 SMM Detection Methodology

```
# CHIPSEC SMM detection

python chipsec_util.py smi scan

python chipsec_util.py smm info

python chipsec_util.py smm compress

python chipsec_main.py -m common.smm

python chipsec_util.py smm smram_lock
```

Znane podatności w SMI handlerach:

- CVE-2020-24489 – Intel – SMI handler buffer overflow
- CVE-2025-4275 – “Hydroph0bia” – Insyde H2O firmware; relevant to OEM notebook fleets
- Vendor-specific SMM callout issues – track via INTEL-SA / AMD-SB advisories per platform

11.4 SMM Indicators of Compromise (IoC)

Indicator	Detection
Unexpected SMI handler	SMI handler enumeration
SMRAM cache poisoning	SMM integrity check (CHIPSEC)
SW SMI from unprivileged code	I/O port 0xB2 monitoring
SMRAM lock bypass	Register check (SMRR, SMM_ACCESS)

SMM Mitigation:

- Enable SMM page table protection (BIOS setting)
- Lock SMRAM after initialization (D_LCK bit)
- Limit SMI sources (USB, PCIe hotplug)
- Use CHIPSEC to verify SMM integrity regularly

11.5 The 2026 Secure Boot Trust Anchor Transition

Microsoft’s Secure Boot certificates issued in 2011 reach expiry in June 2026. This is the first refresh of the Secure Boot chain of trust since its introduction, and it changes the operational picture for every fleet in scope of this paper.

Why it matters operationally

Certificate	Role	Consequence of expiry without replacement
Microsoft Corporation KEK CA 2011	Key Exchange Key — authorises DB and DBX updates	Device can no longer receive new revocations. The DBX freezes at its current state permanently.
Microsoft Windows Production PCA 2011	Signs the Windows boot chain	Newly signed boot components will not validate on un-updated firmware
Microsoft Corporation UEFI CA 2011	Third-party signing (shim, option ROMs, vendor tooling)	Third-party boot components affected on the same timeline

The core risk is not boot failure — it is revocation freeze. A device whose KEK has expired still boots normally. What it silently loses is the ability to accept future DBX entries. Every bootkit revocation published after that point never reaches the device. The failure mode is invisible from the endpoint.

Assessment checklist

```
# Windows — inspect current trust anchors

[System.Text.Encoding]::ASCII.GetString((Get-SecureBootUEFI KEK).Bytes) -match "2011|2023"

Get-SecureBootUEFI db | Select-Object -ExpandProperty Bytes | Measure-Object -Line

Confirm-SecureBootUEFI


# Linux

efi-readvar -v KEK

efi-readvar -v dbx | wc -l

mokutil --list-enrolled
```

Recommended actions

- Inventory KEK certificate issue year across the fleet — 2011 versus 2023 — before the expiry window closes
- Compare each device's DBX against the current published UEFI revocation list; missing entries indicate a device that stopped receiving updates
- Verify firmware update availability per model; OEM support status determines whether a device can receive the 2023 trust anchors at all
- Treat end-of-support hardware that cannot accept new anchors as a separate, tracked risk population
- Note that TPM presence and BitLocker enrollment do **not** imply correct Secure Boot configuration — verify independently

Interaction with PKfail (CVE-2024-8105): devices shipped with leaked AMI test Platform Keys are compromised at the root of the chain. For those models the anchor transition is an opportunity to replace the PK with a genuine production key, not merely to refresh the KEK.

12 Enterprise Implementation Roadmap

12.1 Phase 1: Assessment (Week 1-2)

- Inventory UEFI versions across fleet – Firmware version report
- Secure Boot status audit – Compliance dashboard
- Identify non-Secure Boot systems – Remediation list
- CHIPSEC scan on high-value systems – Baseline report

12.2 Phase 2: Hardening (Week 3-8)

- Enable Secure Boot enterprise-wide – Critical
- Update dbx with all known revocations – Critical
- Enable TPM + BitLocker (with PCR 0-7) – Critical
- Configure firmware write protection – High
- Disable legacy boot (CSM) – High
- Implement UEFI variable monitoring – Medium

12.3 Phase 3: Continuous Monitoring (Ongoing)

- Firmware version drift detection – Monthly
- Secure Boot attestation – Continuous (via MDE/CrowdStrike)
- CHIPSEC full audit – Quarterly (critical systems)
- UEFI variable baseline comparison – Weekly
- Option ROM scanning – Upon new device enrollment

13 Why GGSEC – Threat Research & Firmware Specialization

GGSEC is a specialized security research laboratory focused exclusively on firmware-level threats, embedded systems auditing, and supply chain risk assessment.

13.1 GGSEC Firmware Toolkit (ggfw)

ggfw is GGSEC's in-house firmware analysis platform, covering SPI acquisition, offline image analysis, and platform security assessment in a single tool. It was developed after the first release of this paper and now serves as the primary instrument behind GGSEC's firmware engagements. The analyses reproduced in §9.4 and §10 were produced with it.

Module	Capability
Offline SPI image analysis	Full-chip image parsing with dump scope assessment — distinguishes a true full-chip dump from a BIOS-region-only capture, and reports which descriptor-declared regions are actually covered
Intel Flash Descriptor decode	FLMAP0/1/2, FCBA/FRBA/FMBA/FISBA decode; per-master read/write permission expansion for Host CPU/BIOS, ME and GbE; automated equivalent of the <code>common.spi_desc</code> access-control check
UEFI firmware volume walker	Top-level and nested FV enumeration with header checksum and bounds validation; FFS inventory including deleted, opaque and executable file counts; NVRAM volumes routed to variable-store analysis rather than an FFS walk
PE/DXE anomaly heuristics	Scores executables on host-API imports, W+X sections, entropy and packaging anomalies. Detects the <i>Windows-subsystem PE embedded in a UEFI-subsystem module</i> pattern common to OEM persistence modules and firmware implants (see §6.4)
PE/TE extraction	Extracts flagged modules for offline YARA/AV triage with CSV and JSON manifests; interesting modules are written with a <code>.HIT</code> extension for filtering
Bounded disassembly	Zydis-backed reachable-code analysis of extracted hits. Resolves string cross-references, so a suspicious string with no reachable XREF is reported as low-confidence rather than as a finding
UEFI variable store analysis	VSS/VSS2 store parsing, record state accounting, offline attribute policy checks, and inventory of Secure Boot targets (PK, KEK, db, dbx, SetupMode, DeployedMode)
ME region assessment	\$FPT detection and blank-ratio analysis; identifies host-denied ME reads (FRAP/BRRA) and blocks reporting of an ME version that cannot be established offline
Live MEI / MKHI	Host-side ME/CSE version query where the offline image is unreadable — <code>ggfw --mei --mei-live-only</code>
Region hash inventory	Per-region SHA-256 across Descriptor, BIOS, ME, GbE, PDR plus content-inferred UEFI payload — the baseline unit for fleet drift detection
Firmware family & CSM analysis	Vendor family inference (AMI, InsydeH2O, Phoenix) with confidence scoring; legacy reset-vector and boot-block inspection for CSM-capable platforms

Design principle — heuristics are triage, not verdicts. `ggfw` separates what it observed from what it concluded. A flagged module is ranked for manual review; the tool does not assert that firmware is clean or malicious. Where offline evidence cannot settle a question — runtime Secure Boot state, ME version behind a blocked read — `ggfw` says so and points at the live check instead of guessing. Findings state their own confidence.

Current release: v0.8.0 Beta. GUID identification uses an LVFS-derived bundled database.

13.2 Supporting Tooling

Tool	Purpose
FindBootServices.py (Ghidra)	Scans DxeCore for EFI Boot/Runtime Services hooks — detects CosmicStrand, MoonBounce and MosaicRegressors style hooking
UEFI Volume Extractor	DXE driver extraction, hash baseline generation, anomaly detection
ME Analyzer Automation	ME region integrity validation, checksum verification, manifest signature validation
SMM Integrity Scanner	SMI handler enumeration and SMM memory analysis

13.5 Methodology & Limitations

13.5.1 Methodology

This paper draws on vendor advisories, peer-reviewed and industry security research, conference proceedings, and first-party firmware analysis conducted by GGSEC. CVE identifiers, malware attributions and persistence mechanisms have been cross-checked against primary vendor and researcher sources rather than secondary reporting. Where the research community has not reached consensus on attribution, that uncertainty is stated rather than resolved editorially.

13.5.2 Limitations

- Offensive implementation detail — exploit code, SPI write procedures, bootkit loader internals — is deliberately omitted. This document is written for defenders, firmware auditors and security architects.
- Firmware-level attribution is technically difficult. Attributions reflect community consensus at the time of writing and should be treated as provisional.
- Heuristic detection findings, including those produced by GGSEC tooling, indicate items warranting manual review. They are not malware verdicts.
- Offline firmware analysis cannot establish runtime state. Secure Boot enablement, ME/CSE version and active service status require live host queries.
- CVE status information is time-sensitive. Verify against vendor advisories before operational use.

14 Appendix & References

14.1 Quick Reference Card – Windows Detection Commands

```
Confirm-SecureBootUEFI

Get-UEFIVariable -Name "BootOrder"

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-CodeIntegrity/Operational'} | Where-Object {$_.Id -eq 3076}

Get-WmiObject -Class Win32_BIOS
```

14.2 Quick Reference Card – Linux Detection Commands

```
sbctl status

mokutil --sb-state

ls /sys/firmware/efi/efivars/

dmesg | grep -i "secure boot\|uefi"
```

14.3 Critical CVEs to Track

CVE	Description	Status
CVE-2022-21894	Baton Drop — Secure Boot bypass in Windows Boot Manager (exploited by BlackLotus)	Patched Jan 2022; revocation via KB5025885 chain
CVE-2023-24932	Secure Boot bypass; Microsoft fix required manual opt-in activation	Patched May 2023 — verify enforcement is actually enabled
CVE-2020-10713	BootHole — GRUB2 <code>grub.cfg</code> parser buffer overflow	Patched (dbx update)
CVE-2024-8105	PKfail — leaked AMI test Platform Keys across 800+ device models	Vendor-dependent; audit PK provenance per device
CVE-2023-40238	LogoFAIL — boot logo image parser flaws in DXE (class of CVEs)	Patch pending (vendor dependent)
CVE-2024-7344	Signed UEFI application permitting unsigned code execution; abused by HybridPetya	Revoked Microsoft Patch Tuesday, Jan 2025
CVE-2025-4275	Hydroph0bia — Insyde H2O UEFI firmware	Vendor advisory; relevant to OEM notebooks

14.4 References

1. ESET Research / Smolár, M. (2023). "BlackLotus UEFI bootkit: Myth confirmed." WeLiveSecurity, 1 March 2023.
2. Microsoft Security Response Center. (2023). "Guidance for investigating attacks using CVE-2022-21894: The BlackLotus campaign."
3. NSA. (2023). "BlackLotus Mitigation Guide" (CSI, June 2023).
4. Binarly REsearch. (2024). "PKfail: Untrusted Platform Keys undermine Secure Boot" (CVE-2024-8105).
5. Kamluk, V., Belov, S., Sacco, A. (2014). "Absolute Backdoor Revisited." Black Hat USA.
6. Eclipsium. (2023). "Supply Chain Risk from Gigabyte App Center Backdoor."
7. Kaspersky. (2021). "MoonBounce: the dark side of UEFI firmware."
8. Microsoft Security Response Center. (2023). "CVE-2022-21894 Secure Boot Security Feature Bypass Vulnerability."
9. MITRE ATT&CK. (2024). "Bootkit (T1542.003)" and "System Firmware (T1542.001)."
10. Binarly Research. (2023). "LogoFAIL: Image parser vulnerabilities in UEFI DXE drivers."
11. Quarkslab. (2020). "The BootHole vulnerability: GRUB2 bootloader compromise."

12. Google Project Zero. (2020). "CVE-2020-10713: GRUB2 BootHole."
13. Intel Corporation. (2023). "CHIPSEC: Platform Security Assessment Framework."
14. National Institute of Standards and Technology. (2018). "NIST SP 800-193: Platform Firmware Resiliency Guidelines."
15. US-CERT. (2023). "AA23-165A: BlackLotus UEFI Bootkit."

Maciej Gojny · GG Advanced IT Security · ggsec.de
GGSEC Research · Version 1.1 · July 2026 · TLP:WHITE — Public Release